

@isaacs/brace-expansion A hybrid CJS/ESM TypeScript fork of brace-expansion. Brace expansion, as known from sh/bash, in JavaScript. 		Example <pre>import { expand } from '@isaacs/brace-expansion' expand('file-{a,b,c}.jpg') // => ['file-a.jpg', 'file-b.jpg', 'file-c.jpg'] expand('-v{.,}') // => ['-v', '-v.', '-v.']</pre>	<pre>expand('file{0..2}.jpg') // => ['file0.jpg', 'file1.jpg', 'file2.jpg'] expand('file-{a..c}.jpg') // => ['file-a.jpg', 'file-b.jpg', 'file-c.jpg'] expand('file{2..0}.jpg') // => ['file2.jpg', 'file1.jpg', 'file0.jpg']</pre>
1 8 <pre>/-?\\d+\\.\\.\\.?-?\\d+(\\.\\.\\.?-?\\d+)?\$/ // {x..y[.incr]} A numeric sequence from x to y inclusive, with optional increment. If x or y start with a leading 0, all the numbers will be padded to have equal length. Negative numbers and backwards iteration work too. // {x..y[.incr]}</pre>	2 7 <pre>const expanded = expand(str) Return an array of all possible and valid expansions of str. If none are found, [str] is returned. Valid expansions are: /\\(\\.\\.\\.?)+\\.\\.\\.?\$/ // {a,b,...} A comma separated list of options, like {a,b} or {a,b,c} or {a,}.</pre>	3 9 <pre>import { expand } from '@isaacs/brace-expansion' expand('{{A..C},{a..c}}') // => ['A', 'B', 'C', 'a', 'b', 'c'] expand('ppp{,config,oe,conf}') // => ['ppp', 'pppconfig', 'pppoe', 'pppoeconf']</pre> API	4 5 <pre>expand('file{0..4..2}.jpg') // => ['file0.jpg', 'file2.jpg', 'file4.jpg'] expand('file-{a..e..2}.jpg') // => ['file-a.jpg', 'file-c.jpg', 'file-e.jpg'] expand('file{00..10..5}.jpg') // => ['file00.jpg', 'file05.jpg', 'file10.jpg']</pre>
An alphabetic sequence from x to y inclusive, with optional increment. x and y must be exactly one character, and if given, incr must be a number. For compatibility reasons, the string \${} is not eligible for brace expansion.		11 14	12 13
6 16	10 15	11 14	12 13